



# TomoClub Content Management & GitHub Publishing Guide

---

This guide provides detailed, step-by-step instructions for content creators and developers on how to add new **Podcasts**, **Educational Hall Articles**, and **Blogs** to the TomoClub website, and how to publish those changes to GitHub without needing any agent or AI assistance.

## Table of Contents

---

1. Prerequisites
2. Managing Podcasts
3. Method A: Semi-Automated (Recommended)
4. Method B: Fully Manual
5. Managing Educational Hall Articles
6. Method A: Semi-Automated (Recommended)
7. Method B: Fully Manual
8. Managing Blogs
9. Method A: Semi-Automated (Recommended)
10. Method B: Fully Manual
11. Publishing to GitHub
12. Method A: Using Git Command Line (CLI)
13. Method B: Using GitHub Desktop (GUI)
14. Method C: Using GitHub Web Interface

# Prerequisites

---

- **Text Editor:** A modern code or text editor such as **VS Code**, Sublime Text, or Notepad++.
- **Python 3:** Installed and added to your system's PATH variable (required for running the automation scripts).
- **Git:** Installed on your system (if using the command line for GitHub publishing).

## 1. Managing Podcasts

---

The website features the **The Learning Shift Podcast** on the homepage and provides an interactive player in [podcast-player.html](#).

### Method A: Semi-Automated (Recommended)

This approach uses Python scripts to fetch metadata from YouTube, update the data files, and inject the podcast into the homepage.

#### Step 1: Add the YouTube Link

1. Open [generate\\_html.py](#) in your text editor.
2. Locate the `urls` multi-line string at the top of the file.
3. Append your new YouTube URL to the bottom of the list on a new line (e.g., `https://youtu.be/lV1Aho8vthg`). Save the file.

#### Step 2: Add Player Metadata

1. Open [update\\_js\\_metadata.py](#) in your editor.
2. Locate the `new_podcasts` list at the top of the file.
3. Add a new tuple for your podcast episode at the top of the list in this format: `python ("YOUTUBE_VIDEO_ID", "Speaker Name", "Release Date")` *Example:* `python`

```
new_podcasts = [ ("lv1Aho8vthg", "Priya Darbari", "April 24, 2026"), #
Existing entries... ]
```

4. Save the file and run it using the command: `bash python update_js_metadata.py`  
This will automatically append the detailed structure to `podcasts_data.js`.

### Step 3: Run the Main Update Script

Run the main script that pulls YouTube durations/titles and updates the homepage:

```
python update_all.py
```

This script will execute `fetch_podcasts.py` and `generate_html.py` to rebuild the homepage grid in `index.html`.

## Method B: Fully Manual

If you prefer not to use Python, you can update the files manually.

### Step 1: Update the JavaScript Data

1. Open `podcasts_data.js`.
2. Add a new key-value pair under the `podcastsData` object using the YouTube Video ID as the key: 

```
javascript "YOUTUBE_VIDEO_ID": { "title": "Episode Title",
"speaker": "Speaker Name", "date": "Release Date", "duration": "MM:SS",
"thumbnail":
"https://img.youtube.com/vi/YOUTUBE_VIDEO_ID/maxresdefault.jpg",
"description": "<p>Episode description paragraph.</p>" },
```

### Step 2: Update the Homepage HTML Grid

1. Open `index.html`.
2. Locate the `<div class="grid-3" id="podcast-grid">` section (around line 2411).
3. Insert a new `<a>` card at the top of the grid: 

```
html <a href="podcast-player.html?
id=YOUTUBE_VIDEO_ID" class="glass-card podcast-card" data-
timestamp="UNIX_TIMESTAMP"> <div style="position: relative; width: 100%;
```

```
padding-top: 56.25%; border-radius: 12px; overflow: hidden; margin-bottom:
1rem;">  <div style="position: absolute; top: 0;
left: 0; width: 100%; height: 100%; background: rgba(0,0,0,0.2); display:
flex; align-items: center; justify-content: center;"> <i data-lucide="play-
circle" style="color: white; width: 48px; height: 48px; opacity: 0.8;"></i>
</div> </div> <div style="display: flex; gap: 0.5rem; margin-bottom: 0.5rem;
font-size: 0.75rem; color: var(--teal); font-weight: 700; text-transform:
uppercase; letter-spacing: 0.05em;"> <span>Release Date</span>
<span>•</span> <span>MM:SS</span> </div> <h3 style="font-size: 1.4rem;
margin-bottom: 0.75rem;">Episode Title | The Learning Shift #Number</h3> <p
style="font-size: 0.95rem; color: var(--text-muted); margin-bottom: 1.5rem;
line-height: 1.5;">Explore deep insights into the future of education and AI
literacy.</p> <span class="btn btn-secondary" style="margin-top: auto;
padding: 0.6rem 1.2rem; font-size: 0.9rem; align-self: flex-start;">Read
More <i data-lucide="arrow-right" style="width: 16px; height: 16px;"></i>
</span> </a>
```

## 2. Managing Educational Hall Articles

The **Education Hall** showcases stories from school leaders and educators in [index.html](#) and generates full-page articles under the `/articles/` folder.

### Method A: Semi-Automated (Recommended)

#### Step 1: Add the HTML Content

1. Open `articles_data.js` in your editor.
2. Under the `articlesData` object, add a new key representing your new article (e.g., `"article_9"`), and paste the HTML content of the article as a string value:

```
javascript "article_9": "<h3>Section Header</h3><p>Your paragraph text
here...</p><div style=\"text-align: center; margin: 2rem 0;\"><img
```

```
src=\"https://url-to-image.jpg\" style=\"max-width: 100%; border-radius: 8px;\" /></div>\"
```

## Step 2: Configure Article Metadata

1. Open `generate_article_pages.py`.
2. Locate the `article_metadata` dictionary near the top of the file.
3. Add a new configuration block for the new article using the matching key (e.g., `"article_9"`):
 

```
python "article_9": { "title": "How Your Leader Built Trust in a Tough Environment", "date": "Month Day, Year", "category": "Leadership", "slug": "custom-article-slug-name", "cover": "https://www.tomoclub.org/wp-content/uploads/2026/02/example.jpg" }
```

## Step 3: Generate the Pages

Run the generator script:

```
python generate_article_pages.py
```

This script will automatically download the cover and content images, save them inside the `articles/images/` directory, update image links in your content, and output a complete `index.html` under `articles/custom-article-slug-name/`.

## Step 4: Add Card to the Homepage

1. Open `index.html`.
2. Search for `<div class="grid-3 animate-on-scroll" id="articles-grid">` (around line 2164).
3. Insert the card element inside this container:
 

```
html <a href="articles/custom-article-slug-name/" class="glass-card" style="text-decoration: none; color: inherit; display: block; transition: transform 0.3s ease; padding: 0; overflow: hidden;" target="_blank"> <div style="background: var(--card-gradient); height: 100%; display: flex; flex-direction: column;">  <div style="padding: 2rem; display: flex; flex-direction: column; flex-grow: 1;"> <h3 style="margin: 1rem 0;
```

```
font-size: 1.25rem; font-weight: 600;" class="article-title">How Your Leader
Built Trust in a Tough Environment</h3> <span class="btn btn-secondary btn-
read-more">Read More <i data-lucide="arrow-right"></i></span> </div> </div>
</a>
```

## Method B: Fully Manual

### Step 1: Setup Folder and HTML

1. In the `articles/` folder, create a new directory using your slug name (e.g., `articles/custom-article-slug-name` ).
2. Duplicate the `index.html` from an existing article folder (e.g., `articles/why-tomoclub-exists/index.html` ) and paste it into your new directory.
3. Open the new `index.html` and customize the following parts:
4. Document `<title>`
5. Meta tags for description, Open Graph, and Twitter
6. Main heading `<h1>`
7. Date and Category badges
8. Content inside `<div class="article-content">`

### Step 2: Store Images

1. Save your cover image inside `articles/images/` named as `custom-article-slug-name-cover.png` .
2. Update the cover image `` path inside your article's `index.html` .

### Step 3: Link on the Main Page

1. Follow Step 4 of the Semi-Automated instructions above to insert the card inside `index.html` .

## 3. Managing Blogs

Blogs are located inside the `blog/` folder.

### Method A: Semi-Automated (Recommended)

#### Step 1: Add Blog to the Generation Script

1. Open `generate_new_blogs.py`.
2. Locate the `blogs` list at the bottom of the script.
3. Add a new dictionary object representing the new blog: 

```
python { "slug": "new-blog-slug-name", "title": "Your Awesome Blog Title", "category": "Category Name", "desc": "A short sentence summary of the post used for SEO metadata and card preview.", "content": "" <p class="lead" style="font-size: 1.5rem; font-weight: 600; color: var(--navy); margin-bottom: 3rem; line-height: 1.5;"> Welcome to the lead paragraph. </p> <p>This is the standard body text of the blog.</p> <h2>Important Heading</h2> <p>Further details and list items.</p> "" }
```

#### Step 2: Run the Generation Script

Execute the script to create the blog post folder and file structure:

```
python generate_new_blogs.py
```

This will compile the content using the master styling template and place the resulting page in `blog/new-blog-slug-name/index.html`.

#### Step 3: Add to the Homepage Blog Grid

1. Open `index.html`.
2. Find the `<div id="blog" class="page">` element (around line 5048).
3. Under the inner `<div class="grid-3">`, insert a new blog card: 

```
html <a href="blog/new-blog-slug-name/" class="glass-card" style="text-decoration: none; color: inherit; display: block; transition: transform 0.3s ease; padding: 0; overflow: hidden;" target="_blank"> <div style="background:
```

```

var(--card-grad-gold)); height: 100%; display: flex; flex-direction:
column;">  <div style="padding:
2rem 2rem 2rem; display: flex; flex-direction: column; flex-grow: 1;"> <span
style="font-size: 0.85rem; color: #D97706; font-weight: 600; text-transform:
uppercase;">Category Name</span> <h3 style="margin: 1rem 0; font-size:
1.25rem; font-weight: 600;">Your Awesome Blog Title</h3> <p style="font-
size: 0.95rem; color: var(--text-muted); margin-bottom: 1.5rem; line-height:
1.5; display: -webkit-box; -webkit-line-clamp: 3; -webkit-box-orient:
vertical; overflow: hidden;">A short sentence summary of the post used for
SEO metadata and card preview.</p> <span class="btn btn-secondary btn-read-
more" style="margin-top: auto;">Read More <i data-lucide="arrow-right"></i>
</span> </div> </div> </a>

```

## Method B: Fully Manual

### Step 1: Create Directory and File

1. In the `blog/` folder, create a new directory using your slug name (e.g., `blog/new-blog-slug-name` ).
2. Duplicate the `index.html` from an existing blog folder (e.g., `blog/ai-implementation-in-k12-schools/index.html` ) and paste it into your new directory.
3. Open your new `index.html` and update:
4. Title tag and SEO meta descriptions.
5. `<h1>` title heading, Category label, and date badges.
6. The content inside `<div class="article-content">` .

### Step 2: Store Card Images

1. Save your cover/hero image as `hero.png` inside your newly created folder ( `blog/new-blog-slug-name/hero.png` ).
2. Follow Step 3 of the Semi-Automated instructions above to insert the card inside `index.html` .

## 4. Publishing to GitHub

---

Once you have modified or created the content files locally, follow these methods to publish them live to your GitHub repository.

### Method A: Using Git Command Line (CLI)

Open your terminal (PowerShell on Windows, or Terminal on macOS/Linux), navigate to your project directory `tom/`, and run the following command sequence:

1. **Verify changed files:** `bash git status`
2. **Stage all changes** (adds all new folders, HTML changes, and image additions): `bash git add .`
3. **Commit the changes with a clear message:** `bash git commit -m "Add new podcast episode, blog post, and educational hall article"`
4. **Push changes to GitHub:** `bash git push origin main` (Replace `main` with `master` if your default branch is named `master`)

### Method B: Using GitHub Desktop (GUI)

If you prefer a visual interface, download and open **GitHub Desktop**:

1. **Open the repository:** Add the local folder containing the TomoClub files ( `tom` ) into GitHub Desktop.
2. **Review your changes:** The app will list all modified and new files on the left sidebar. Click on any file to view a diff of changes.
3. **Stage files:** Ensure the checkboxes next to the files you want to push are checked.
4. **Write a Commit Message:** In the bottom-left corner, enter a summary title (e.g., `Add new content updates` ) and optional description.
5. **Commit:** Click the blue **Commit to main** button.
6. **Push:** Click **Push origin** in the top bar to upload the changes to GitHub.

## Method C: Using GitHub Web Interface

For small text adjustments or adding simple files directly through a web browser:

1. Navigate to your repository on [GitHub](#).
2. **To edit an existing file** (e.g., `index.html`):
3. Click on the file, click the pencil icon (**Edit this file**) in the top right.
4. Make your edits, scroll down, write your commit message, and click **Commit changes**.
5. **To add a new file or folder:**
6. Click **Add file** -> **Upload files**.
7. Drag and drop your new folder (e.g. `blog/your-new-post/` ) or file.
8. Write a commit message and click **Commit changes**.